

Lab 5 - DLX Execute

Nate Herbst

A02307138

Nathan Walker

A02364124

Introduction

This project required implementing the execute stage, the third of five stages in the DLX processor pipeline. This stage contains several components including: an ALU, a branch checker ("zero" block), two 2-to-1 multiplexers, and pipeline registers. The execute stage receives decoded instruction fields, register data, the sign-extended immediate, and the PC+1 value from the decode stage. It performs arithmetic/logical operations via the ALU, evaluates branch and jump conditions, and passes the ALU result, branch enable signal, and forwarded data through pipeline registers to the next stage. With this stage complete, the processor now self-drives its own branches and jumps without testbench intervention.

Procedure

Modular Systems:

We built each submodule to match the blocks in our pipeline block diagram:

- ALU — A combinational case statement over all 49 opcodes. Handles signed/unsigned add, subtract, AND, OR, XOR, logical/arithmetic shifts, all six set-comparisons (signed and unsigned), and pass-through of branch/jump target addresses.
- Branch Checker — Evaluates BEQZ, BNEZ (conditional on rs1), and unconditional jumps (J, JR, JAL, JALR) that always output take_branch = '1'.
- Two MUXes — MUX1 selects between rs1_data and the extended PC (currently hardwired to rs1 since we use absolute addressing). MUX2 selects between rs2_data (R-type) and sign_ext_imm (I-type) based on the opcode.
- Execute wrapper (execute.vhd) — Instantiates the above submodules with MUX control logic and five pipeline registers (ALU result, branch enable, rs2 data, instruction, and rd_addr) at the stage output.

We created individual testbenches for the ALU and branch checker to verify each piece in isolation (See Figures 1-2). We then integrated the execute stage into ``DLX_Processor.vhd`` alongside the existing fetch and decode stages, with the execute's branch_en and ALU result feeding back to the fetch stage's PC MUX.

We also fixed the decode stage's register file to register the read outputs synchronously, aligning them with the other pipelined signals, and added destination register address selection logic (I-type → bits[20:16], R-type → bits[15:11], JAL/JALR → R31).

Challenges and Solutions:

The primary challenge was data hazards. Since our pipeline has no forwarding or stalling yet, a register written by one instruction isn't available for reading until two cycles later. Our professor confirmed this is expected and will be addressed in a future lab. The workaround for testing was to insert two NOP instructions after every register-writing instruction in the .mif file, giving the pipeline time to complete writes before subsequent reads.

A second challenge was getting the MUX control logic and ALU behavior correct for branches and jumps. Since our DLX uses absolute (not relative) addressing, the ALU doesn't compute branch targets — it simply passes them through. For BEQZ/BNEZ and J/JAL, the target comes from the immediate field (data_in2). For JR/JALR, the target comes from rs1 (data_in1). The branch checker handles the decision of whether to take the branch, while the ALU ensures the correct address appears at its output for the fetch stage.

A third challenge involved signed vs. unsigned operations in the ALU. Following the lecture, addition and subtraction produce identical hardware regardless of “signedness” (the beauty of two's complement). Shifts treat the shift amount as unsigned but use signed arithmetic for SRA (replicating the sign bit). The set-comparison instructions are the only ones where signedness truly matters, requiring explicit casting to signed or unsigned types before comparing.

Results

The ALU testbench verified all arithmetic, logical, shift, comparison, and branch/jump pass-through operations produce correct outputs. The branch checker testbench confirmed that BEQZ branches only when rs1=0, BNEZ branches only when rs1≠0, J/JR/JAL/JALR always assert take_branch, and all other opcodes produce '0'. The integrated three-stage DLX_Processor simulation showed instructions flowing correctly through fetch → decode → execute, with branch_en going high for branch/jump instructions and the ALU result reflecting computed values. As expected, two garbage instructions appear in the pipeline after each taken branch before the PC redirect takes effect.

Figures

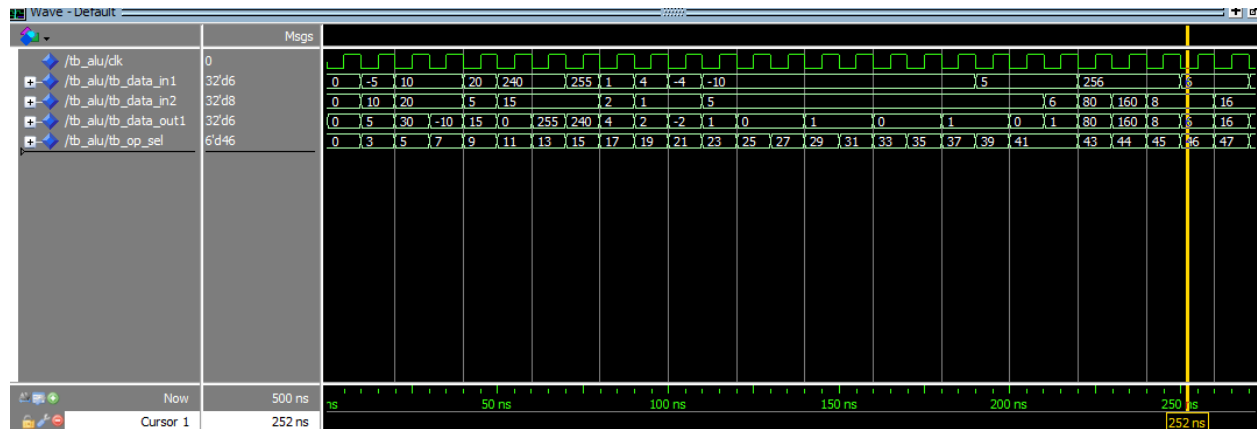


Figure 1: `tb_ALU.vhd` Testbench simulation

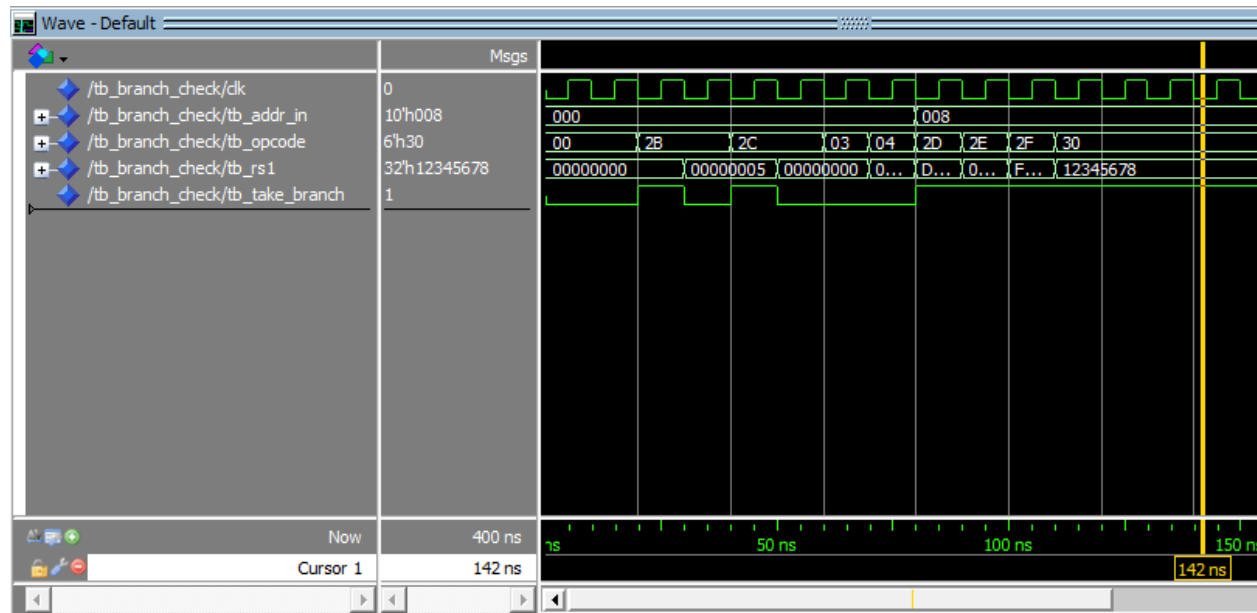


Figure 2: `tb_branch_check.vhd` Testbench simulation

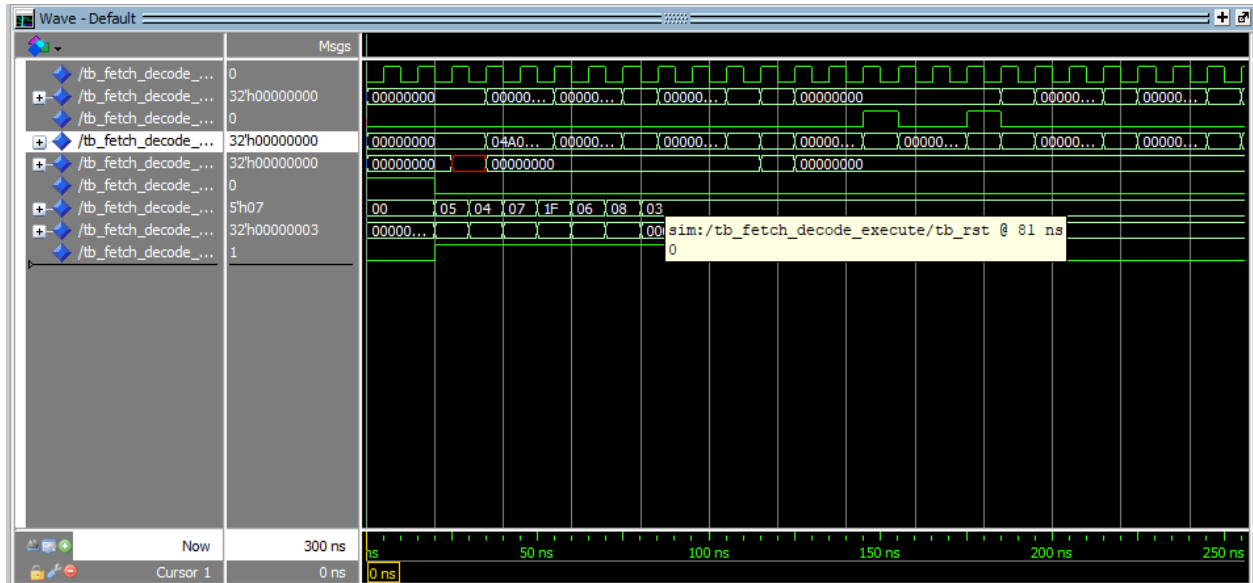


Figure 3: `tb_fetch_decode_execute.vhd` Testbench simulation

Conclusion

This lab brought the processor to the point where it can actually perform computation rather than just preparing to do so. Implementing the execute stage reinforced the importance of keeping pipeline register boundaries consistent across all signals, and highlighted the impact of data and control hazards on correctness. The NOP-insertion workaround demonstrated why forwarding and stalling are essential for real pipeline performance. These lessons will directly apply as we add the memory and writeback stages in future labs.

Appendix

DLX_Processor.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;

library work;
use work.fetch_pkg.all;

entity DLX_Processor is
    generic (
        WIDTH          : integer := 10;
        INSTR_WIDTH    : integer := 32
    );
    port (
        clk             : in  std_logic;
        rst             : in  std_logic;

        -- Writeback Interface (Driven by TB for now/Writeback stage later)
        write_addr_in   : in  std_logic_vector(4 downto 0);
        write_data_in   : in  std_logic_vector(31 downto 0);
        write_en_in     : in  std_logic;

        -- Execute stage outputs (directly visible for TB verification)
        branch_en_out   : out  std_logic;
        alu_result_out  : out  std_logic_vector(INSTR_WIDTH-1 downto
0);
        rs2_data_out    : out  std_logic_vector(INSTR_WIDTH-1 downto
0);
        exec_instr_out  : out  std_logic_vector(INSTR_WIDTH-1 downto
0);
        exec_rd_addr_out : out std_logic_vector(4 downto 0)
    );
end entity DLX_Processor;

architecture structural of DLX_Processor is
    -- Fetch -> Decode signals
    signal internal_pc_inc : std_logic_vector(WIDTH-1 downto 0);
```

```

signal internal_instr    :    std_logic_vector(INSTR_WIDTH-1 downto 0);

-- Decode -> Execute signals
signal dec_instruction   :    std_logic_vector(INSTR_WIDTH-1 downto 0);
signal dec_rs1_data      :    std_logic_vector(INSTR_WIDTH-1 downto 0);
signal dec_rs2_data      :    std_logic_vector(INSTR_WIDTH-1 downto 0);
signal dec_imm32         :    std_logic_vector(INSTR_WIDTH-1 downto
0);
signal dec_rd_addr       :    std_logic_vector(4 downto 0);
signal dec_pc_inc        :    std_logic_vector(WIDTH-1 downto 0);

-- Execute outputs / feedback
signal exec_branch_en    :    std_logic;
signal exec_alu_result   :    std_logic_vector(INSTR_WIDTH-1 downto 0);
signal exec_rs2_data     :    std_logic_vector(INSTR_WIDTH-1 downto
0);
signal exec_instr        :    std_logic_vector(INSTR_WIDTH-1 downto
0);
signal exec_rd_addr      :    std_logic_vector(4 downto 0);

-- Jump address derived from ALU result (for branches/jumps)
signal jump_addr         :    std_logic_vector(WIDTH-1 downto 0);

begin

-- Route execute outputs to top-level ports
branch_en_out            <= exec_branch_en;
alu_result_out           <= exec_alu_result;
rs2_data_out             <= exec_rs2_data;
exec_instr_out           <= exec_instr;
exec_rd_addr_out         <= exec_rd_addr;

-- Jump address is the lower bits of the ALU result
jump_addr <= exec_alu_result(WIDTH-1 downto 0);

-- Fetch Stage
fetch_inst: fetch
    generic map (
        N => WIDTH,
        M => INSTR_WIDTH

```

```

    )
    port map (
        jump_addr    => jump_addr,
        pc_select    => exec_branch_en,
        rst          => rst,
        clk          => clk,
        decode_addr  => internal_pc_inc,
        instruction  => internal_instr
    );

-- Decode Stage
decode_inst: entity work.decode
    generic map (
        FUNC_WIDTH => 6,
        ADDR_WIDTH => 5,
        DATA_WIDTH => 32
    )
    port map (
        clk            => clk,
        rst            => rst,
        instruction_in  => internal_instr,
        pc_inc         => internal_pc_inc,
        wb_data        => write_data_in,
        wb_addr        => write_addr_in,
        wb_en          => write_en_in,
        rs1_data       => dec_rs1_data,
        rs2_data       => dec_rs2_data,
        sign_ext_imm   => dec_imm32,
        rd_addr_out    => dec_rd_addr,
        pc_inc_out     => dec_pc_inc,
        instruction_out => dec_instruction
    );

-- Execute Stage
execute_inst: entity work.execute
    generic map (
        DATA_WIDTH => INSTR_WIDTH,
        PC_WIDTH    => WIDTH
    )
    port map (

```

```

        clk          => clk,
        rst          => rst,
        instruction   => dec_instruction,
        pc_inc        => dec_pc_inc,
        rs1_data      => dec_rs1_data,
        rs2_data      => dec_rs2_data,
        sign_ext_imm  => dec_imm32,
        rd_addr_in    => dec_rd_addr,
        Branch_en     => exec_branch_en,
        ALU_result     => exec_alu_result,
        rs2_data_out   => exec_rs2_data,
        instr_out      => exec_instr,
        rd_addr_out    => exec_rd_addr
    );

end architecture structural;

```

execute.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.decode_reg_pkg.all;

entity execute is
    generic(
        DATA_WIDTH  : integer := 32;
        PC_WIDTH      : integer := 10
    );
    port (
        clk          : in      std_logic;
        rst          : in      std_logic;

        instruction   : in      std_logic_vector(DATA_WIDTH-1 downto 0);
        pc_inc        : in      std_logic_vector(PC_WIDTH-1 downto 0);
    );
end entity execute;

```



```

    rs1_data      : in std_logic_vector(DATA_WIDTH-1 downto 0);
    rs2_data      : in      std_logic_vector(DATA_WIDTH-1 downto
0);

    sign_ext_imm: in      std_logic_vector(DATA_WIDTH-1 downto 0);
    rd_addr_in   : in  std_logic_vector(4 downto 0);

    Branch_en    : out std_logic;
    ALU_result   : out      std_logic_vector(DATA_WIDTH-1 downto 0);
    rs2_data_out : out std_logic_vector(DATA_WIDTH-1 downto 0);
    instr_out    : out std_logic_vector(DATA_WIDTH-1 downto 0);
    rd_addr_out  : out std_logic_vector(4 downto 0)
);
end entity execute;

architecture structural of execute is
    signal opcode      : std_logic_vector(5 downto 0);
    signal ALU_out     : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal branch_reg  : std_logic_vector(0 downto 0);
    signal branch_out  : std_logic_vector(0 downto 0);
    signal q1          : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal q2          : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal ext_pc      : std_logic_vector(DATA_WIDTH-1 downto 0);

    -- MUX control signals
    signal mux1_sel : std_logic; -- '1' = PC (for JAL/JALR), '0' = rs1
    signal mux2_sel : std_logic; -- '1' = immediate (I-type), '0' = rs2
(R-type)

begin

    -- Zero-extend PC to DATA_WIDTH
    ext_pc(PC_WIDTH-1 downto 0) <= pc_inc;
    ext_pc(DATA_WIDTH-1 downto PC_WIDTH) <= (others => '0');

    opcode <= instruction(31 downto 26);

    -- MUX1 Control: Always select rs1_data. PC not needed in ALU for
absolute addressing.

    -- PC is forwarded through pipeline separately for JAL/JALR writeback
to R31 (future lab).

```

```

-- For JR/JALR, rs1 contains the jump target, so it MUST pass through
to the ALU.
mux1_sel <= '0';

-- MUX2 Control: Select immediate for I-type instructions, rs2 for
R-type
-- When S='1', MUX outputs A (sign_ext_imm). When S='0', MUX outputs B
(rs2_data).
-- I-type: LW, SW, all Immediate ALU ops, branches, JAL, JALR
-- R-type: register-register ALU ops (ADD, SUB, AND, OR, XOR, shifts,
compares)
mux2_sel <= '0' when (opcode = OP_ADD or opcode = OP_ADDU or
opcode = OP_SUB or opcode = OP_SUBU or
opcode = OP_AND or opcode = OP_OR or
opcode = OP_XOR or
opcode = OP_SLL or opcode = OP_SRL or opcode =
OP_SRA or
opcode = OP_SLT or opcode = OP_SLTU or
opcode = OP_SGT or opcode = OP_SGTU or
opcode = OP_SLE or opcode = OP_SLEU or
opcode = OP_SGE or opcode = OP_SGEU or
opcode = OP_SEQ or opcode = OP_SNE)
else '1';

MUXXY1 : entity work.MUX
generic map(
    N=>32
)
port map(
    A=>ext_pc,
    B=>rs1_data,
    S=>mux1_sel,
    OUTPUT=>q1
);

MUXXY2 : entity work.MUX
generic map(
    N=>32
)
port map(

```

```

        A=>sign_ext_imm,
        B=>rs2_data,
        S=>mux2_sel,
        OUTPUT=>q2
    );

ALU_inst      :    entity work.ALU
    generic map(
        DATA_WIDTH=>DATA_WIDTH,
        OP_CODE_WIDTH=> 6
    )
    port map(
        data_in1=>q1,
        data_in2=>q2,
        op_sel=>opcode,
        data_out1=>ALU_out
    );

ALU_out_reg :    entity work.reggi
    generic map(
        N => DATA_WIDTH
    )
    port map(
        data_in=>ALU_out,
        rst=>rst,
        clk=>clk,
        data_out=>ALU_result
    );

branch_check_inst      :    entity work.branch_check
    generic map(
        ADDR_WIDTH=>PC_WIDTH
    )
    port map(
        addr_in=>pc_inc,
        rs1=>rs1_data,
        opcode=>opcode,
        take_branch=>branch_reg(0)
    );

```

```

branch_check_reg      :    entity work.reggi
    generic map(
        N=>1
    )
    port map(
        data_in=>branch_reg,
        rst=>rst,
        clk=>clk,
        data_out=>branch_out
    );

```

```

Branch_en <= branch_out(0);

```

```

rs2_reg :    entity work.reggi
    generic map(
        N=>DATA_WIDTH
    )
    port map(
        data_in=>rs2_data,
        rst=>rst,
        clk=>clk,
        data_out=>rs2_data_out
    );

```

```

instr_reg  :    entity work.reggi
    generic map(
        N=>DATA_WIDTH
    )
    port map(
        data_in=>instruction,
        rst=>rst,
        clk=>clk,
        data_out=>instr_out
    );

```

```

rd_addr_reg :    entity work.reggi
    generic map(
        N=>5
    )
    port map(

```

```

        data_in=>rd_addr_in,
        rst=>rst,
        clk=>clk,
        data_out=>rd_addr_out
    );

end architecture structural;

```

ALU.vhd

sign_extend.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity sign_extend is
    port (
        input_data  : in  std_logic_vector(15 downto 0);
        output_data : out std_logic_vector(31 downto 0)
    );
end entity sign_extend;

architecture behavior of sign_extend is
begin
    output_data <= std_logic_vector(resize(signed(input_data), 32));
end architecture behavior;

```

branch_check.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity branch_check is
    generic(

```

```

        ADDR_WIDTH : integer := 10
    );
    port(
        addr_in      : in      std_logic_vector(ADDR_WIDTH-1 downto 0);
        rs1           : in      std_logic_vector(31 downto 0);
        opcode        : in      std_logic_vector(5 downto 0);

        take_branch   : out std_logic
    );
end entity branch_check;

architecture behavioral of branch_check is
begin
    process(addr_in, rs1, opcode)
        variable v_opcode : integer;
        variable v_rs1    : integer;
    begin

        v_opcode := to_integer(unsigned(opcode));
        v_rs1    := to_integer(unsigned(rs1));

        if(v_opcode = 43) then
            if(v_rs1 = 0) then
                take_branch <= '1';
            else
                take_branch <= '0';
            end if;
        elsif(v_opcode = 44) then
            if(v_rs1 /= 0) then
                take_branch <= '1';
            else
                take_branch <= '0';
            end if;
        elsif(v_opcode = 45 or v_opcode = 46 or v_opcode = 47 or v_opcode =
48) then
            -- J, JR, JAL, JALR: always take the jump
            take_branch <= '1';
        else
            take_branch <= '0';
        end if;
    end process;
end architecture;

```

```
end process;

end architecture behavioral;
```

tb_fetch_decode_execute.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_fetch_decode_execute is
end entity tb_fetch_decode_execute;

architecture behavioral of tb_fetch_decode_execute is

    component DLX_Processor
        generic (
            WIDTH          : integer := 10;
            INSTR_WIDTH    : integer := 32
        );
        port (
            clk             : in  std_logic;
            rst             : in  std_logic;
            write_addr_in   : in  std_logic_vector(4 downto 0);
            write_data_in   : in  std_logic_vector(31 downto 0);
            write_en_in     : in  std_logic;
            branch_en_out   : out std_logic;
            alu_result_out  : out std_logic_vector(INSTR_WIDTH-1 downto
0);
            rs2_data_out    : out std_logic_vector(INSTR_WIDTH-1 downto
0);
            exec_instr_out  : out std_logic_vector(INSTR_WIDTH-1 downto
0);
            exec_rd_addr_out : out std_logic_vector(4 downto 0)
        );
    end component DLX_Processor;

    -- Clock
    signal clk : std_logic := '0';
```

```

constant CLK_PERIOD : time := 10 ns;

-- Inputs
signal tb_rst      : std_logic := '1';
signal tb_wb_addr  : std_logic_vector(4 downto 0) := (others => '0');
signal tb_wb_data  : std_logic_vector(31 downto 0) := (others => '0');
signal tb_wb_en    : std_logic := '0';

-- Execute stage outputs
signal tb_branch_en : std_logic;
signal tb_alu_result : std_logic_vector(31 downto 0);
signal tb_rs2_data  : std_logic_vector(31 downto 0);
signal tb_exec_instr : std_logic_vector(31 downto 0);
signal tb_exec_rd_addr : std_logic_vector(4 downto 0);

begin

-- Instantiate the DLX Processor (Fetch + Decode + Execute)
uut : DLX_Processor
    generic map (
        WIDTH      => 10,
        INSTR_WIDTH => 32
    )
    port map (
        clk          => clk,
        rst          => tb_rst,
        write_addr_in => tb_wb_addr,
        write_data_in => tb_wb_data,
        write_en_in   => tb_wb_en,
        branch_en_out => tb_branch_en,
        alu_result_out => tb_alu_result,
        rs2_data_out  => tb_rs2_data,
        exec_instr_out => tb_exec_instr,
        exec_rd_addr_out => tb_exec_rd_addr
    );

-- Clock process
clk_process : process
begin
    clk <= '0';

```



```

    wait for CLK_PERIOD / 2;
    clk <= '1';
    wait for CLK_PERIOD / 2;
end process;

-- =====
-- FACTORIAL PROGRAM IN ROM (factorial_NOops.mif):
-- NOP-padded to avoid data hazards (2 NOPs after every
-- register-writing instruction and every branch/jump).
--
-- 000: 04A00001  LW   R5, n(R0)                [writes R5]
-- 001: 00000000  NOP
-- 002: 00000000  NOP
-- 003: 10800001  ADDI R4, R0, 1                [writes R4]
-- 004: 00000000  NOP
-- 005: 00000000  NOP
-- 006: 081F0000  SW   R4, f(R0)
-- 007: 10E5FFFF  ADDI R7, R5, -1              [writes R7]
-- 008: 00000000  NOP
-- 009: 00000000  NOP
-- 00A: ACE00000  BEQZ R7, done(02C)           [branch]
-- 00B: 00000000  NOP (delay slot)
-- 00C: 00000000  NOP (delay slot)
-- 00D: BC000000  JAL  factorial(016)          [jump, writes R31]
-- 00E: 00000000  NOP (delay slot)
-- 00F: 00000000  NOP (delay slot)
-- 010: 20A50001  SUBI R5, R5, 1                [writes R5]
-- 011: 00000000  NOP
-- 012: 00000000  NOP
-- 013: B4000000  J    loop(007)               [jump]
-- 014: 00000000  NOP (delay slot)
-- 015: 00000000  NOP (delay slot)
-- 016: 04C00000  LW   R6, f(R0)                [writes R6]
-- 017: 00000000  NOP
-- 018: 00000000  NOP
-- 019: 0D002800  ADD  R8, R0, R5              [writes R8]
-- 01A: 00000000  NOP
-- 01B: 00000000  NOP
-- 01C: 10600000  ADDI R3, R0, 0                [writes R3]
-- 01D: 00000000  NOP

```

```

-- 01E: 00000000 NOP
-- 01F: 0C633000 ADD R3, R3, R6 [writes R3]
-- 020: 00000000 NOP
-- 021: 00000000 NOP
-- 022: 21080001 SUBI R8, R8, 1 [writes R8]
-- 023: 00000000 NOP
-- 024: 00000000 NOP
-- 025: B1000000 BNEZ R8, multiply_loop(01F) [branch]
-- 026: 00000000 NOP (delay slot)
-- 027: 00000000 NOP (delay slot)
-- 028: 081F0000 SW R3, f(R0)
-- 029: B800001F JR R31 [jump]
-- 02A: 00000000 NOP (delay slot)
-- 02B: 00000000 NOP (delay slot)
-- 02C: B4000000 J done(02C) [jump]
-- 02D: 00000000 NOP (delay slot)
-- 02E: 00000000 NOP (delay slot)
--
-- NOTES:
-- 1. Branches/jumps are SELF-DRIVEN by execute. No testbench
-- control of PC needed. Only writeback is TB-driven.
--
-- 2. Jump target addresses in the MIF are encoded as 0, so all
-- branches/jumps redirect PC to 000 (program restart).
--
-- 3. Without a memory or writeback stage, register values only
-- change when the TB writes them. The NOPs prevent timing
-- misalignment in the pipeline, but the actual register
-- contents depend entirely on TB writeback signals.
--
-- 4. Pipeline timing (3-stage, approx):
-- Instruction at addr N appears at exec_instr_out ~N+2 cycles
-- after pipeline start. Branch_en effect reaches fetch 1 cycle
-- after it appears at execute output.
--
-- 5. Expected linear flow (R7 nonzero, BEQZ not taken):
-- 000-006: setup (LW, NOPs, ADDI, NOPs, SW)
-- 007-009: ADDI R7, NOPs
-- 00A-00C: BEQZ (not taken), NOPs
-- 00D: JAL → branch_en='1' → PC redirects to 000

```

```

--      00E-00F: NOPs (garbage in pipeline, harmless)
--      Then repeats from 000.
-- =====

stm_process : process
begin
    -- =====
    -- PHASE 1: RESET
    -- =====

    tb_rst <= '1';
    tb_wb_en <= '0';
    tb_wb_addr <= (others => '0');
    tb_wb_data <= (others => '0');
    wait for CLK_PERIOD * 2;
    tb_rst <= '0';

    -- =====
    -- PHASE 2: PRE-POPULATE REGISTERS
    -- Write register values immediately after reset.
    -- Pipeline is starting but the first instructions that
    -- read these registers don't reach decode for several
    -- cycles (NOPs give us plenty of margin).
    --
    -- Without a writeback stage, the ALU computes results
    -- but can't write them to registers. The TB must provide
    -- all register values that instructions depend on.
    -- =====

    -- R5 = 5 (n, the factorial input)
    -- Used by: ADDI R7,R5,-1 at addr 007
    tb_wb_en <= '1';
    tb_wb_addr <= "00101";
    tb_wb_data <= x"00000005";
    wait for CLK_PERIOD;

    -- R4 = 1 (initial f value)
    -- Used by: SW R4,f(R0) at addr 006
    tb_wb_addr <= "00100";
    tb_wb_data <= x"00000001";
    wait for CLK_PERIOD;

```

```

-- R7 = 4 (n-1, nonzero so BEQZ at 00A is NOT taken)
-- Used by: BEQZ R7 at addr 00A
tb_wb_addr <= "00111";
tb_wb_data <= x"00000004";
wait for CLK_PERIOD;

-- R31 = 16 (return address, simulates JAL writeback)
-- Used by: JR R31 at addr 029 (unreachable in first pass)
tb_wb_addr <= "11111";
tb_wb_data <= x"00000010";
wait for CLK_PERIOD;

-- R6 = 1 (f loaded from memory)
-- Used by: ADD R3,R3,R6 at addr 01F (unreachable in first pass)
tb_wb_addr <= "00110";
tb_wb_data <= x"00000001";
wait for CLK_PERIOD;

-- R8 = 5 (multiply loop counter = n)
-- Used by: SUBI R8,R8,1 at addr 022 (unreachable in first pass)
tb_wb_addr <= "01000";
tb_wb_data <= x"00000005";
wait for CLK_PERIOD;

-- R3 = 0 (accumulator)
-- Used by: ADD R3,R3,R6 at addr 01F (unreachable in first pass)
tb_wb_addr <= "00011";
tb_wb_data <= x"00000000";
wait for CLK_PERIOD;

tb_wb_en <= '0';

-- =====
-- PHASE 3: FIRST PASS (addresses 000 through 00D)
-- The pipeline runs linearly from 000. With R7=4 (nonzero),
-- BEQZ at 00A is NOT taken. JAL at 00D fires branch_en='1'
-- with target=0, looping back to 000.
--
-- In waveform, verify:

```

```

-- - exec_instr_out shows LW, NOP, NOP, ADDI, NOP, NOP, SW,
--   ADDI, NOP, NOP, BEQZ, NOP, NOP, JAL in sequence
-- - alu_result_out for ADDI R4,R0,1 (addr 003) = 0x00000001
-- - alu_result_out for ADDI R7,R5,-1 (addr 007) = 0x00000004
-- - branch_en_out stays '0' for BEQZ (R7 nonzero)
-- - branch_en_out goes '1' for JAL at addr 00D
-- - After JAL, PC redirects to 000 (2 NOPs flush harmlessly)
--
-- 14 instructions (000-00D) + 2 pipeline fill + 2 delay slots
-- = ~18 cycles minimum. Use 20 for margin.
-- =====
wait for CLK_PERIOD * 20;

-- =====
-- PHASE 4: SECOND PASS (pipeline has restarted at 000)
-- Update registers to simulate progress from first pass.
-- The processor loops back through the same instructions.
-- =====

-- R5 = 4 (decremented n, simulating SUBI R5,R5,1)
tb_wb_en <= '1';
tb_wb_addr <= "00101";
tb_wb_data <= x"00000004";
wait for CLK_PERIOD;

-- R7 = 3 (new n-1 = 4-1)
tb_wb_addr <= "00111";
tb_wb_data <= x"00000003";
wait for CLK_PERIOD;

-- R3 = 5 (accumulator after first multiply: 1 * 5 = 5)
tb_wb_addr <= "00011";
tb_wb_data <= x"00000005";
wait for CLK_PERIOD;

-- R6 = 5 (f updated)
tb_wb_addr <= "00110";
tb_wb_data <= x"00000005";
wait for CLK_PERIOD;

```

```

-- R8 = 4 (new loop counter)
tb_wb_addr <= "01000";
tb_wb_data <= x"00000004";
wait for CLK_PERIOD;

tb_wb_en <= '0';

-- Let second pass run (same flow: 000→00D→loop to 000)
wait for CLK_PERIOD * 20;

-- =====
-- PHASE 5: BEQZ TAKEN PATH
-- Set R7=0 so BEQZ at 00A fires.
-- This tests that the branch checker correctly evaluates
-- BEQZ when rs1=0 and produces branch_en='1'.
-- Target is still 0, so PC goes to 000 either way,
-- but branch_en will go HIGH at the BEQZ instruction
-- instead of waiting for JAL at 00D.
--
-- Verify in waveform:
-- - branch_en_out goes '1' when BEQZ (opcode 0x2B)
--   appears at exec_instr_out
-- =====

tb_wb_en <= '1';
tb_wb_addr <= "00111"; -- R7
tb_wb_data <= x"00000000"; -- R7 = 0 → BEQZ taken
wait for CLK_PERIOD;

tb_wb_en <= '0';

-- Let pipeline run: 000→00A(BEQZ taken)→000
-- BEQZ at 00A, faster loop: ~12 instructions + pipeline
wait for CLK_PERIOD * 20;

-- =====
-- PHASE 6: FINAL OBSERVATION
-- Pipeline continues looping. All branches/jumps go to 0.
-- Let it run a few more cycles for waveform capture.
-- =====

```

```
        wait for CLK_PERIOD * 10;

        wait;
    end process;

end architecture behavioral;
```